application server, and add a special section *<Resource>* into *ROOT.xml*. The latter file holds information on how to access the required database (credentials, IP address, database type and so forth).

```
<Resource
        name="jdbc/MainPool_Level_1"
        auth="Container"
        type="javax.sql.DataSource"
        driverClassName="org.postgresql.Driver"
        url="jdbc:postgresql://warehouse10_level1.
corp:5432/Products_DB"
        username="mega"
        password="pass"
        maxActive="20"
/>
```

The above code declares the information pool section in order to get access to the database.

Every time you run an SQL-query addressed to *MainPool_Level_1* pool, your request will actually be transferred to the *Products_DB* base, where it is stored at *warehouse10_level1.corp* PostgreSQL-server.

Well, now you have a tool that simultaneously fetches data directly from two databases. However, I hope you remember that there exists one unified database (Base3) that accumulates everything, and information from which can't be requested directly because security policy prohibits it. That's why all necessary bits are daily exported into a neutral-looking HTML-page. In this case we have to tune our *cron* to periodically save this page locally, and perform a search based on the product's ID in order to get the arrival date. Then we have to adapt our JSP-code a little to display these statistics. Luckily, this new procedure can be done with the help of regular expressions, the *sed* (stream editor) utility and some shell magic. But more on that in Part Two. END

By: Anton Borisov

The author has specialised in Linux and FOSS technologies for more than a decade. His professional spheres of interest include, but are not limited to, robotics, embedded systems, statistics and algorithmic methods.

Continued from page 64...

file system comes back online after the crash, there is a file system inconsistency typically known as a dangling d-entry. How do file systems avoid/prevent such inconsistencies? The answer lies in what is known as file system logging/journaling, wherein the file system first writes the intent of performing a metadata change to a log/journal, the journal/log is made persistent on the disk, and only after that the actual metadata change is made.

Before actually writing back the metadata changes corresponding to inode and dentry creation, the file system code writes a journal log entry signifying its intention to create the inode and dentry. The file system code is written to the journal for the given file name and it specifies the operations to undo these changes. Only after the log entry has been flushed to disk completely, is the actual metadata changed, and the log entry is marked as committed in the journal. Now if there is a system failure before the completion of the metadata update to disk, when the file system comes back online after the crash, it examines the log entries in the journal. For each log entry that has not been committed, it performs the corresponding undo operations specified, which takes the file system back to a consistent state. Now here is a question for our readers: Should we also undo the incomplete log entries or is it possible to re-do these operations?

My 'must-read book' for this month

This month's must-read book suggestion comes from our reader, Sudha Jayaram. It is 'Understanding the Linux virtual memory manager' by Mel Gorman. This book discusses the internals of the Linux virtual memory manager describing each component of the VM subsystem in detail. It is great to enhance your knowledge of the Linux kernel internals. While the book is slightly dated, since it describes the Linux kernel 2.4.22, it also provides an overview of the changes planned for 2.6. A soft copy of the book is available from *https://www.kernel.org/doc/gorman/pdf/understand.pdf*. Thank you Sudha, for your suggestion.

If you have a favourite programming book/article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful so I can mention it in the column. This would help many readers who want to improve their software skills.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at *sandyasm_AT_yahoo_DOT_com*. Till we meet again next month, happy programming and here's wishing you the very best! END

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working with Hewlett Packard India Ltd. Her interests include compilers, multi-core and storage systems. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training India' at *http://www.linkedin.com/groups?home=&gid=2339182*